JobSheet - Week 6 – Java Stream API

Nama : **Zahra Azkiya Rahmani**

NIM : **251524124**

Kelas : **1D**

Repo GitHub : **https://github.com/yayyzea/1D_251524124_Teknik-Pemrograman-PR-.git**

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W6_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W6_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Java Stream Sorting

Soal Praktikum
Instruksi Lengkapilah bagian kosong pada cuplikan kode di bawah ini dengan ketentuan sebagai berikut: 1. Bagian [1]: Panggil metode untuk mengubah ArrayList menjadi Stream. 2. Bagian [2]: Gunakan metode pengurutan (intermediate operation) yang menerima lambda expression. 3. Bagian [3]: Panggil metode getter yang sesuai untuk membandingkan nama antar objek. 4. Bagian [4]: Gunakan operasi terminal untuk merubah aliran data kembali menjadi objek List. 5. Bagian [5]: Gunakan utility class dan metode yang tepat untuk mengumpulkan hasil akhir ke dalam format List.

Source Code

```
package JavaProblems;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

public class ProblemOne {
  public static void main(String[] args) {
    ArrayList<Employee> list = new ArrayList<>();

    list.add(new Employee("John", 50000));
    list.add(new Employee("Rob", 70000));
    list.add(new Employee("Bob", 40000));
    list.add(new Employee("Alice", 10000));

    // Melakukan proses filtering/sorting menggunakan Stream API
    List<Employee> sortedEmp = list.[1]_____ // Ubah list ke stream
        // Mengurutkan employee berdasarkan nama
        .[2]_____((e1, e2) -> e1.[3]_____().compareTo(e2.getName()))
        // Mengumpulkan hasil akhir
        .[4]_____( [5]_____.toList() );

    for(Employee e : sortedEmp){
      System.out.println(e);
    }
  }
}

// Buatlah class Employee disertai dengan getter getName() dan getSalary(),
termasuk attribute dan method lainnya jika diperlukan!
```

Output yang Diharapkan

```
Employee{name='Alice', salary=50000}
Employee{name='Bob', salary=70000}
Employee{name='John', salary=10000}
Employee{name='Rob', salary=40000}
```

Source Code

1. class ProblemOne

```
package JavaProblems;

/*@author ZAHRA*/

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;
```

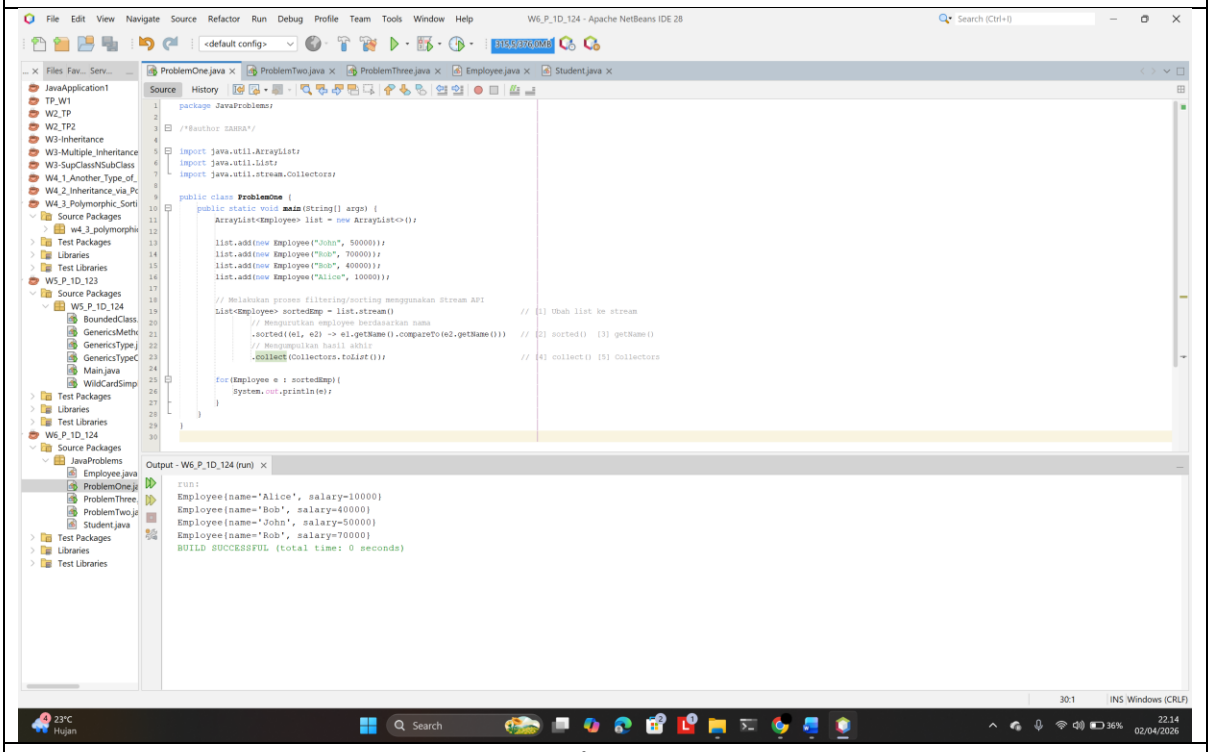
```
public class ProblemOne {
   public static void main(String[] args) {
      ArrayList<Employee> list = new ArrayList<>();

      list.add(new Employee("John", 50000));
      list.add(new Employee("Rob", 70000));
      list.add(new Employee("Bob", 40000));
      list.add(new Employee("Alice", 10000));

      // Melakukan proses filtering/sorting menggunakan Stream API
      List<Employee> sortedEmp = list.stream()                    // [1] Ubah list ke stream
            // Mengurutkan employee berdasarkan nama
            .sorted((e1, e2) -> e1.getName().compareTo(e2.getName()))   // [2] sorted()  [3]
getName()
            // Mengumpulkan hasil akhir
            .collect(Collectors.toList());                        // [4] collect() [5] Collectors

      for(Employee e : sortedEmp){
         System.out.println(e);
      }
   }
}
```

2. class Employee

```
package JavaProblems;

/*@author ZAHRA*/

public class Employee {
   private String name;
   private int salary;

   public Employee(String name, int salary) {
      this.name = name;
      this.salary = salary;
   }

   public String getName() {
      return name;
   }

   public int getSalary() {
      return salary;
   }

   @Override
   public String toString() {
```

```
        return "Employee{name='" + name + "', salary=" + salary + "}";
    }
}
```

Screenshot Hasil



Penjelasan

Soal ini meminta untuk mengurutkan objek Employee di dalam sebuah ArrayList berdasarkan atribut nama secara alfabetis menggunakan Stream API, bukan dengan loop manual (imperatif).

Koleksi harus diubah terlebih dahulu menjadi Stream menggunakan “stream()” agar bisa diproses secara deklaratif. Setelah itu, pipeline Stream dibangun dengan dua tahap:

- Intermediate
  sorted digunakan sebagai intermediate operation untuk mengurutkan elemen dalam stream. Method ini menerima lambda expression (e1, e2) -> e1.getName().compareTo(e2.getName())) sebagai Comparator. getName dipanggil untuk mengambil nilai nama dari objek Employee agar bisa dibandingkan menggunakan compareTo() karena pengurutan dilakukan berdasarkan nama.
  Operasi ini bersifat *lazy*, artinya belum dieksekusi sampai ada terminal operation.
- Terminal
  collect digunakan sebagai terminal operation untuk mengakhiri pipeline stream dan mengumpulkan hasilnya. Tanpa terminal operation, seluruh intermediate operation bersifat lazy dan tidak akan dieksekusi sama sekali.
  Collectors merupakan utility class bawaan Java yang menyediakan method toList(). Dipasangkan dengan collect() agar hasil akhir stream dikumpulkan dalam bentuk List<Employee> sesuai tipe variabel sortedEmp.

Proses pengurutan data koleksi dilakukan secara deklaratif melalui pipeline Stream API. Pemrosesan ini dibagi menjadi operasi intermediate (sorted) yang bersifat lazy (hanya menyusun instruksi pengurutan), dan diakhiri dengan operasi terminal (collect) yang berfungsi memicu eksekusi seluruh instruksi untuk menghasilkan keluaran output akhir.
Nama Teman Hal yang Dibantu (Opsional)

No. 2 Multi-Level Sorting dengan Stream

Soal Praktikum

Skenario
Anda diminta untuk mengurutkan daftar objek Student. Aturan pengurutannya adalah:
1. Urutkan berdasarkan Nama secara alfabetis (tanpa mempedulikan huruf besar/kecil).
2. Jika ada nama yang sama, maka urutkan berdasarkan ID dari yang terkecil ke terbesar.
 	
Instruksi Pengerjaan
Lengkapilah bagian kosong pada kode di bawah ini dengan ketentuan:
1. Bagian [1]: Panggil fungsi untuk memulai operasi Stream pada koleksi.
2. Bagian [2]: Gunakan metode Stream untuk melakukan pengurutan elemen.
3. Bagian [3]: Gunakan metode dari kelas wrapper Integer untuk membandingkan dua nilai ID secara aman.
4. Bagian [4]: Gunakan metode String yang tepat untuk membandingkan dua string tanpa memperhatikan case (huruf besar/kecil) untuk menghasilkan nilai integer perbandingan.
5. Bagian [5]: Panggil operasi terminal untuk mengumpulkan hasil akhir ke dalam sebuah List.
 	
Instruksi
package JavaProblems;
 	
import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
 	
public class ProblemTwo {
  public static void main(String[] args) {
 	
  List<Student> students = Arrays.asList(
  new Student(101, "Joe"),
  new Student(103, "Zulkifli"),
  new Student(102, "Riza"),
  new Student(104, "Alice"),
  new Student(105, "Joshua")
  );
 	
  List<Student> ans = students.[1]_____ // Membuka stream
  .[2]_____((s1, s2) -> { // Melakukan pengurutan
 	
  // Logika: Jika nama sama, bandingkan ID
  if (s1.getName().equalsIgnoreCase(s2.getName())) {
  return [3]_____.compare(s1.getId(), s2.getId());

```
            }
            // Jika nama berbeda, bandingkan Nama
            else {
              return s1.getName().[4]__________(s2.getName());
            }
          })
          .[5]__________(Collectors.toList()); // Mengumpulkan hasil

    for (Student student : ans) {
      System.out.println(student);
    }
  }
}

// Buatlah class Student disertai dengan attribute dan method lainnya yang
diperlukan!
```

Output yang Diharapkan

```
Student{id=104, name='Alice'}
Student{id=101, name='Joe'}
Student{id=105, name='Joshua'}
Student{id=102, name='Riza'}
Student{id=103, name='Zulkifli'}
```

Source Code

1. Class ProblemTwo

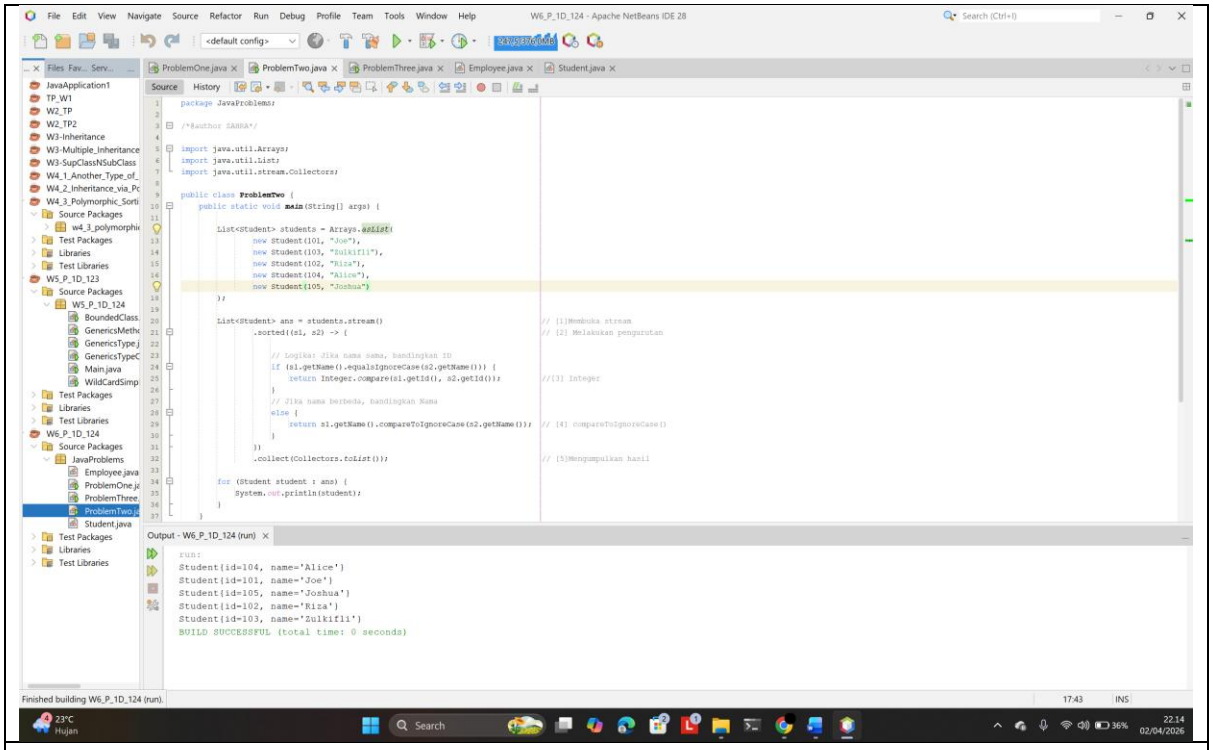
```
package JavaProblems;

/*@author ZAHRA*/

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;

public class ProblemTwo {
  public static void main(String[] args) {

    List<Student> students = Arrays.asList(
        new Student(101, "Joe"),
        new Student(103, "Zulkifli"),
        new Student(102, "Riza"),
        new Student(104, "Alice"),
        new Student(105, "Joshua")
    );

    List<Student> ans = students.stream()                    // [1]Membuka stream
        .sorted((s1, s2) -> {                        // [2] Melakukan pengurutan
```

```
            // Logika: Jika nama sama, bandingkan ID
            if (s1.getName().equalsIgnoreCase(s2.getName())) {
              return Integer.compare(s1.getId(), s2.getId());       //[3] Integer
            }
            // Jika nama berbeda, bandingkan Nama
            else {
              return     s1.getName().compareToIgnoreCase(s2.getName());         //     [4]
compareToIgnoreCase()
            }
          })
          .collect(Collectors.toList());                          // [5]Mengumpulkan hasil

      for (Student student : ans) {
        System.out.println(student);
      }
    }
}
```

2. class Student

```
package JavaProblems;

/*@author ZAHRA*/

public class Student {
   private int id;
   private String name;

   public Student(int id, String name) {
      this.id = id;
      this.name = name;
   }

   public int getId() {
      return id;
   }

   public String getName() {
      return name;
   }

   @Override
   public String toString() {
      return "Student{id=" + id + ", name='" + name + "'}";
   }
}
```

Screenshot Hasil



Penjelasan

Soal ini memberikan daftar objek Student yang perlu diurutkan berdasarkan dua kriteria sekaligus: pertama berdasarkan nama secara alfabetis tanpa memperhatikan huruf besar/kecil, dan jika ada nama yang sama, urutkan berdasarkan ID terkecil. Pengurutan berlapis seperti ini tidak bisa dilakukan dengan satu perbandingan sederhana.

- Membuka Stream: Metode stream() dipanggil untuk mengubah objek koleksi agar dapat memproses aliran data.
- Intermediate
  sorted digunakan untuk mengurutkan elemen. Karena aturan pengurutannya berlapis (nama kemudian ID), dibutuhkan lambda dengan blok {} yang memuat logika percabangan if-else.
  Jika nama berbeda → bandingkan menggunakan compareToIgnoreCase() agar tidak membedakan huruf besar/kecil.
  Jika nama sama → bandingkan ID menggunakan Integer.compare(s1.getId(), s2.getId()) yang merupakan cara aman membandingkan dua nilai integer.
- Terminal
  collect memicu eksekusi seluruh pipeline dan mengumpulkan hasilnya ke dalam List sesuai yang diminta oleh Collectors.toList().

Stream API memfasilitasi pengurutan data multitingkat (multi-level) yang kompleks di dalam satu pipeline pemrosesan tanpa memerlukan logika perulangan manual (imperatif). Aturan komparasi berlapis tersebut dilakukan di dalam intermediate operation (sorted) dan dieksekusi hanya ketika terminal operation (collect) dipanggil pada akhir pipeline.

Nama Teman Hal yang Dibantu (Opsional)

No. 3 Finding the Nth Largest Element

Soal Praktikum

Skenario

Anda memiliki sebuah array integer primitif. Anda diminta untuk mencari elemen terbesar ke-3 dari array tersebut menggunakan Java 8 Stream. Tahapannya adalah: mengubah array menjadi stream, mengubah tipe data primitif menjadi objek agar bisa diurutkan secara kustom, mengurutkan dari yang terbesar, melompati elemen tertentu, dan mengambil hasilnya.

Instruksi

Isilah bagian rumpang pada kode di bawah ini dengan ketentuan:

1. Bagian [1]: Panggil metode dari kelas Arrays untuk mengubah array menjadi IntStream.
2. Bagian [2]: Gunakan metode untuk mengubah stream primitif (int) menjadi stream objek (Integer) agar bisa menggunakan Comparator.
3. Bagian [3]: Gunakan operasi untuk melompati (skip) sejumlah elemen pertama.
4. Bagian [4]: Gunakan operasi terminal untuk mengambil elemen pertama yang tersedia setelah proses skipping.
5. Bagian [5]: Gunakan metode untuk memberikan nilai default jika stream ternyata kosong.

```
package JavaProblems;

import java.util.Arrays;

public class ProblemThree {
   public static void main(String[] args) {
      int arr[] = {12, 4, 3, 1, 9, 657};
      int n = 3; // Mencari elemen terbesar ke-3

      int ans = [1]__________.stream(arr)
            .[2]__________()                    // Konversi ke Stream<Integer>
            .sorted((a, b) -> Integer.compare(b, a))    // Urutkan descending
            .[3]__________(n - 1)                // Lewati 2 elemen pertama
            .[4]__________()                    // Ambil elemen yang tersisa
            .[5]__________(0);                  // Nilai default jika tidak ada

      System.out.println("Elemen terbesar ke-3 adalah: " + ans);
   }
}
```

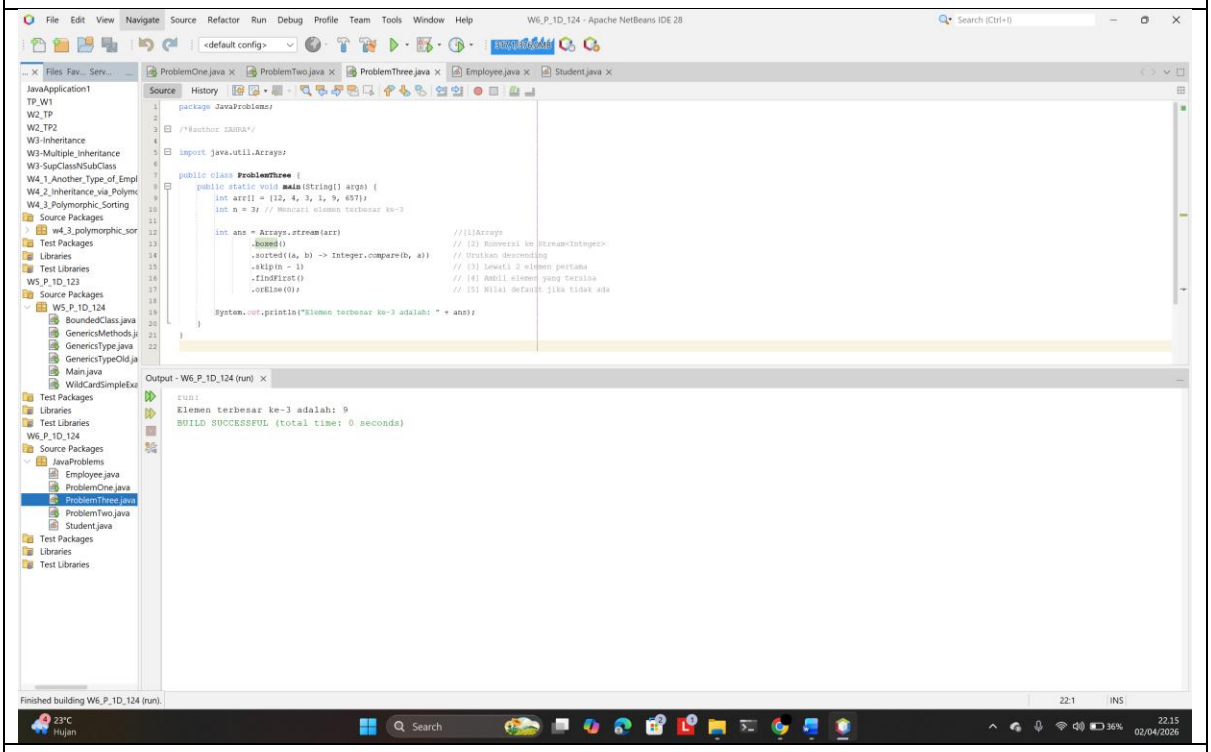
Output yang Diharapkan

The 3rd largest element is: 9

Source Code

```
package JavaProblems;

/*@author ZAHRA*/

import java.util.Arrays;

public class ProblemThree {
   public static void main(String[] args) {
      int arr[] = {12, 4, 3, 1, 9, 657};
      int n = 3; // Mencari elemen terbesar ke-3

      int ans = Arrays.stream(arr)                    //[1]Arrays
            .boxed()                              // [2] Konversi ke Stream<Integer>
            .sorted((a, b) -> Integer.compare(b, a))    // Urutkan descending
            .skip(n - 1)                          // [3] Lewati 2 elemen pertama
            .findFirst()                          // [4] Ambil elemen yang tersisa
            .orElse(0);                           // [5] Nilai default jika tidak ada

      System.out.println("Elemen terbesar ke-3 adalah: " + ans);
   }
}
```

Screenshot Hasil



Penjelasan

Soal ini memberikan array integer primitif {12, 4, 3, 1, 9, 657} dan meminta untuk menemukan elemen terbesar ke-3 menggunakan Stream API. Tantangannya adalah array primitif (int[]) tidak langsung bisa menggunakan Comparator, sehingga perlu dikonversi terlebih dahulu ke tipe objek.

- Arrays dipanggil karena input datanya adalah array primitif int[], bukan koleksi. Kelas Arrays menyediakan method static stream(arr) yang khusus untuk mengubah array primitif menjadi IntStream.
- Intermediate
  boxed dipanggil karena Arrays.stream(int[]) menghasilkan IntStream (stream primitif), sedangkan operasi .sorted() dengan Comparator custom seperti (a, b) -> Integer.compare(b, a) hanya bisa digunakan pada Stream<Integer> (stream objek). Method boxed() melakukan konversi tersebut dengan membungkus setiap nilai int menjadi objek Integer.
  skip digunakan untuk melewati sejumlah elemen pertama setelah stream diurutkan secara descending.
- Terminal
  findFirst digunakan untuk mengambil elemen pertama yang tersisa setelah proses skip. Setelah melewati 2 elemen, elemen pertama berikutnya adalah elemen terbesar ke-3 yaitu 9. Method ini mengembalikan Optional<Integer>, bukan langsung int.
- orElse dipanggil pada hasil Optional dari findFirst() untuk mengekstrak nilai integer di dalamnya. Parameter 0 berfungsi sebagai nilai default yang akan dikembalikan jika stream ternyata kosong, misalnya ketika nilai n lebih besar dari jumlah elemen array.

Manipulasi pada struktur array primitif dapat dilakukan dengan mengonversinya menjadi stream dan melakukan transformasi (pembungkusan) menjadi objek menggunakan metode boxed. Pencarian urutan elemen spesifik dilakukan dengan merangkai intermediate operation (sorted dan skip), yang kemudian dieksekusi oleh terminal operation (findFirst), serta dilengkapi dengan penanganan nilai bawaan (orElse).

Nama Teman Hal yang Dibantu (Opsional)